

Event-Driven Architectures in Elixir

Maciej Kaszubowski

ElixirConfEU 2018

 [mkaszubowski94](#)

Event-Driven Architectures

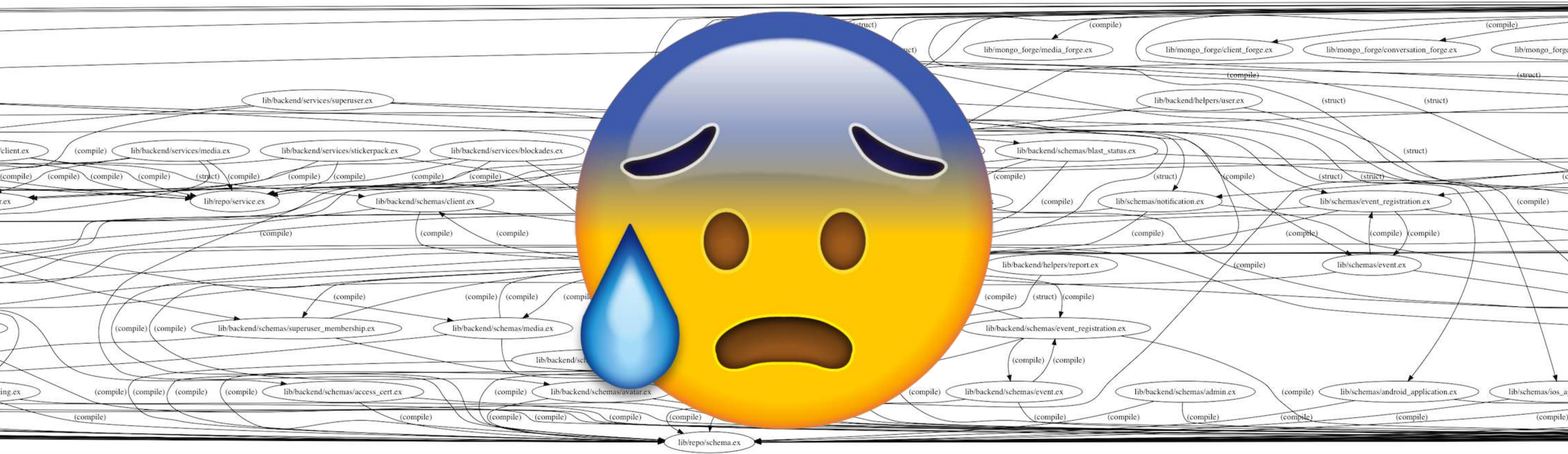
- ▶ Event Sourcing / CQRS
- ▶ Microservices
- ▶ Kafka, RabbitMQ, ...
- ▶ Message passing / OTP

This is not the point



AppUnit

2+ years of Elixir in production



**Building software is a
complex task**

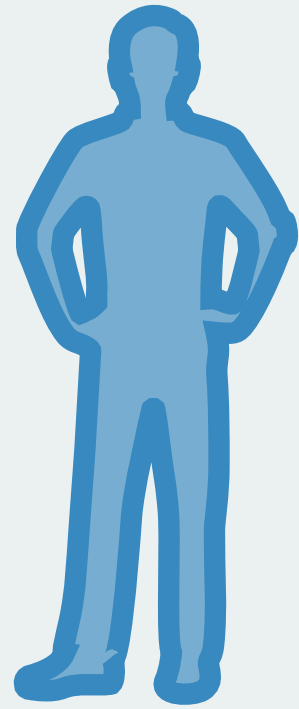
**Changing software
is even harder**

So, this is not a talk about:

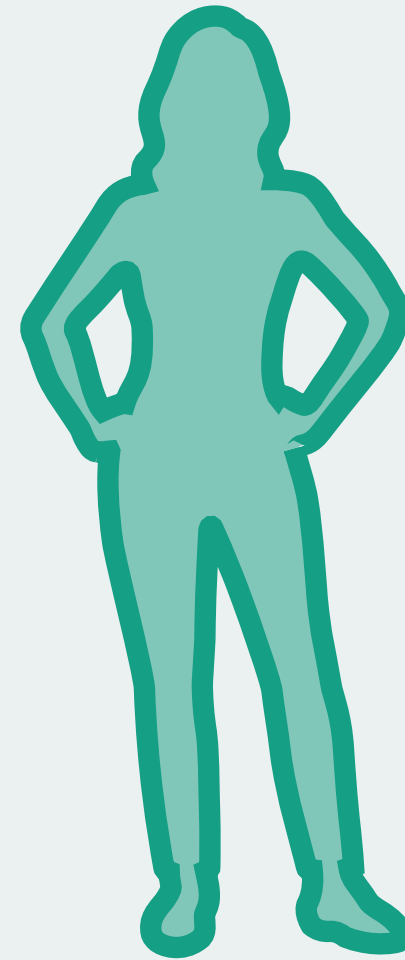
- ▶ Event Sourcing / CQRS
- ▶ Microservices
- ▶ Kafka, RabbitMQ, ...
- ▶ Message passing / OTP

This is a talk about
building small,
independent things

Our project

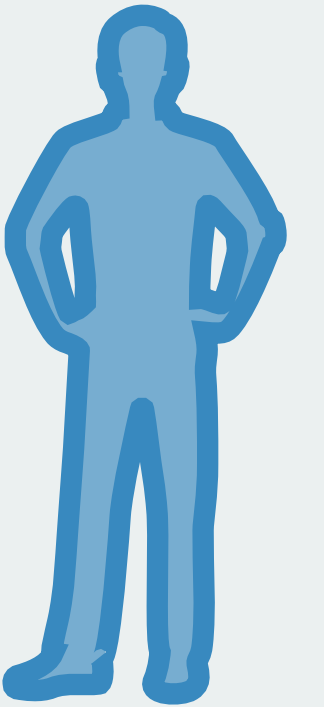


Guest

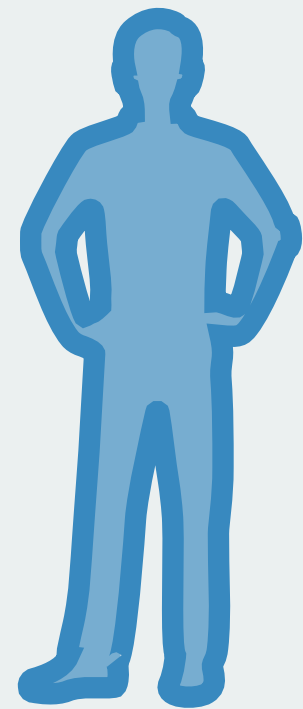


Owner

*Hey, I have a free
apartment!*

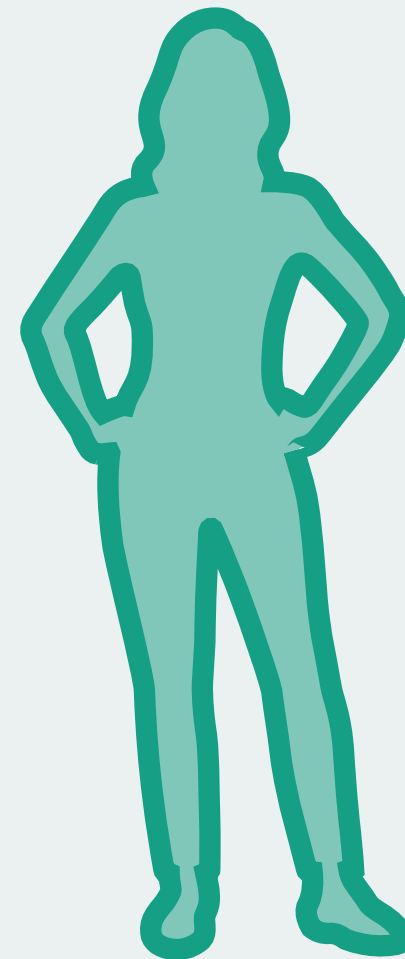
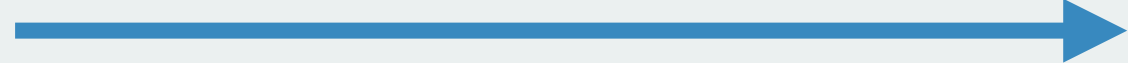


Guest



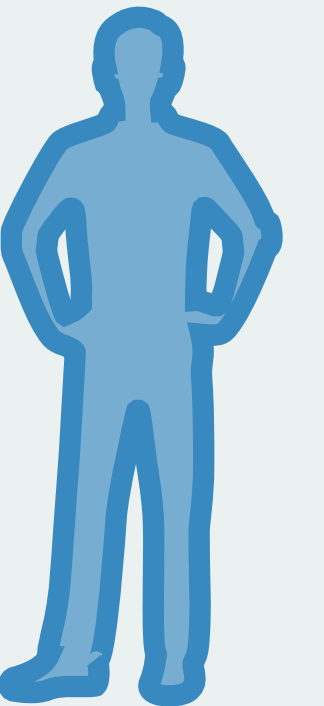
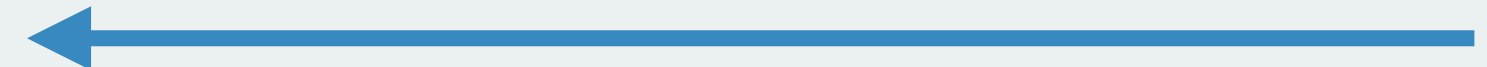
Guest

Can I stay there
next Friday?

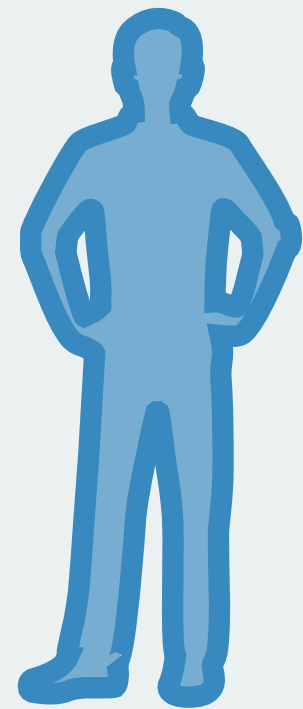


Owner

Cool, can I stay
there next week?

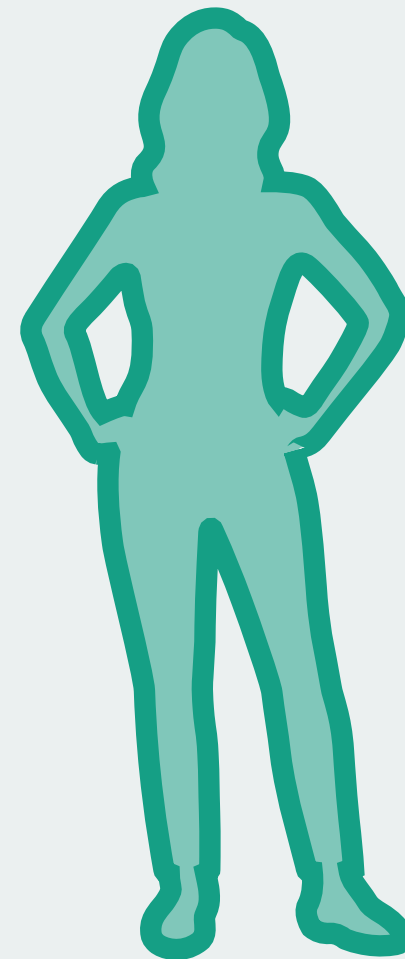


Guest



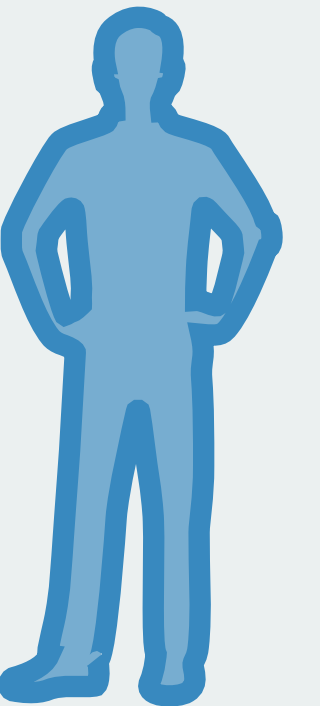
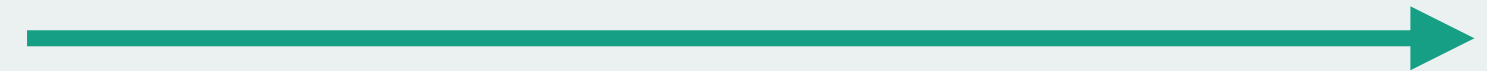
Guest

Sorry, already
taken :(

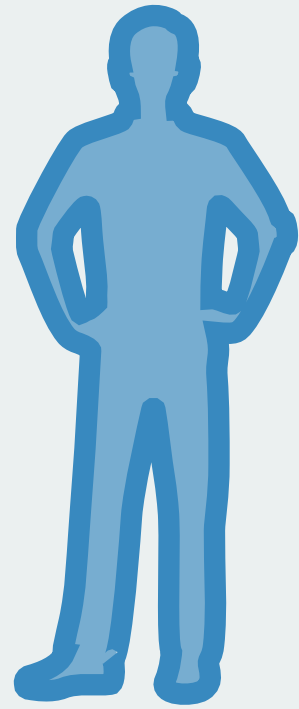


Owner

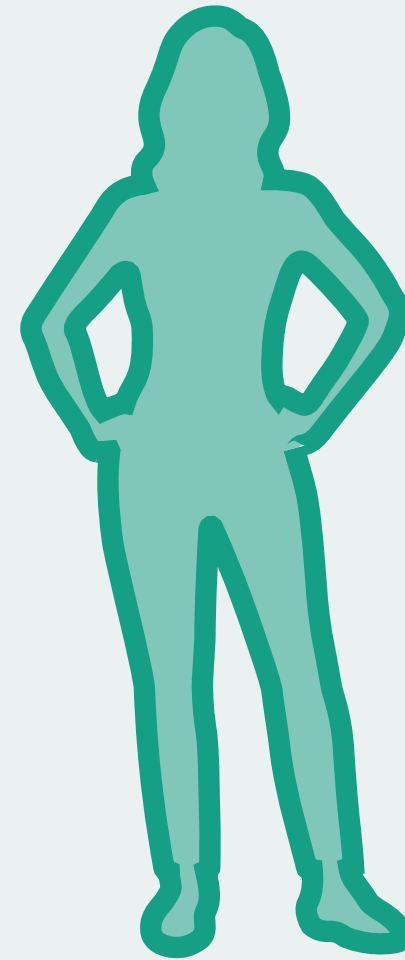
Sure!



Guest

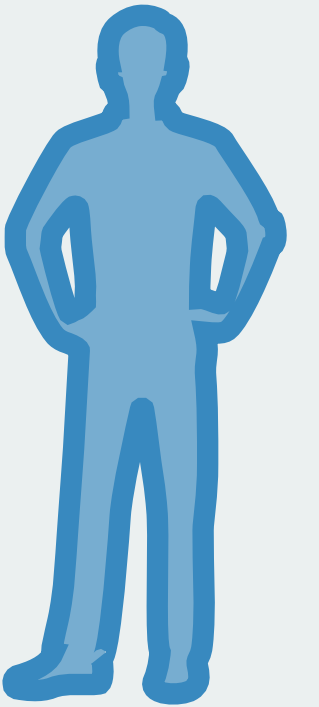
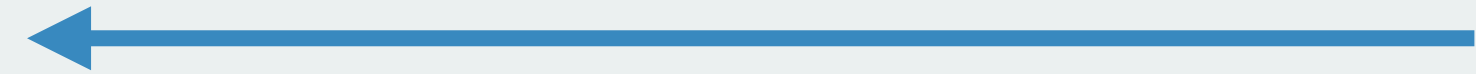


Guest



Owner

\$\$\$



Guest

Let's build this

```
defmodule AirBnbApartmentsContext do
  def create_apartment(params) do
    changeset = Apartment.changeset(%Apartment{}, params)

    case Repo.insert(changeset) do
      {:ok, apartment} → {:ok, apartment}
      {:error, changeset} → {:error, changeset}
    end
  end

  # ...

end
```

```
defmodule AirBnbApartmentsContext do

  # ...

  def fetch_apartments(query_params) do
    Apartment
    |> query_by_location(query_params)
    |> query_by_price(query_params)
    |> ...
    |> Repo.all()
  end
end
```



```
defmodule Airbnb.BookingContext do
  def create_reservation(apartment_id, guest_id,
                        start_date, end_date) do

    params = %{
      apartment_id: apartment_id, guest_id: guest_id,
      start_date: start_date, end_date: end_date
    }

    %Reservation{}
    |> Reservation.changeset(params)
    |> Repo.insert()
  end
end
```

```
defmodule Airbnb.BookingContext do

  def reject_reservation(owner_id, reservation_id) do
    reservation =
      Repo.get(Reservation, reservation_id)
      |> Repo.reload(:apartment)

    if reservation.apartment.owner_id == owner_id do
      Repo.delete(reservation)
    end
  end
end
```

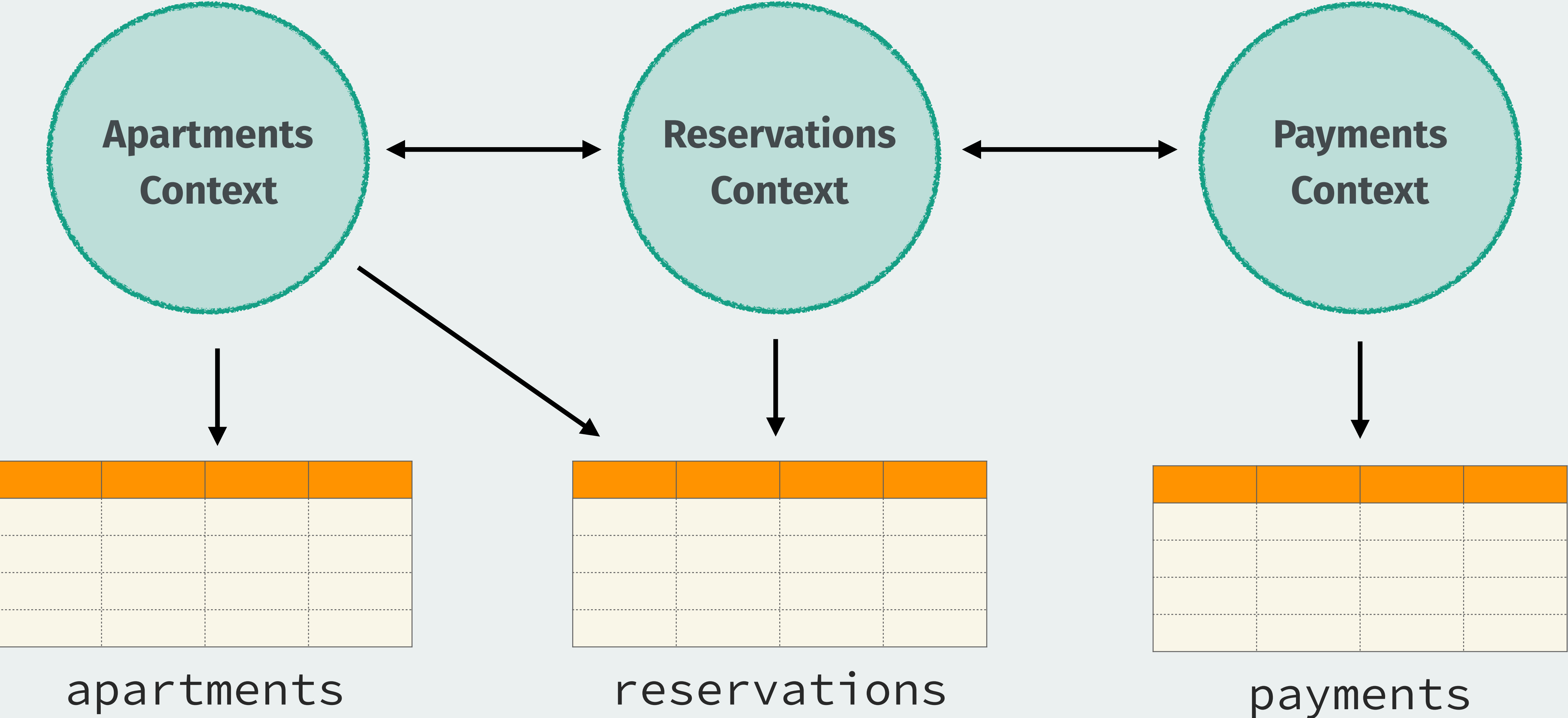
```
defmodule Airbnb.BookingContext do

  def accept_reservation(owner_id, reservation_id) do
    reservation =
      Repo.get(Reservation, reservation_id)
      |> Repo.reload(:apartment)

    if reservation.apartment.owner_id == owner_id do
      reservation
      |> Reservation.changeset(%{accepted: true})
      |> Repo.update()
    end
  end
end
```


Accepting reservation

- ▶ Notify the guest
- ▶ Charge guest's credit card
- ▶ Mark the apartment as taken



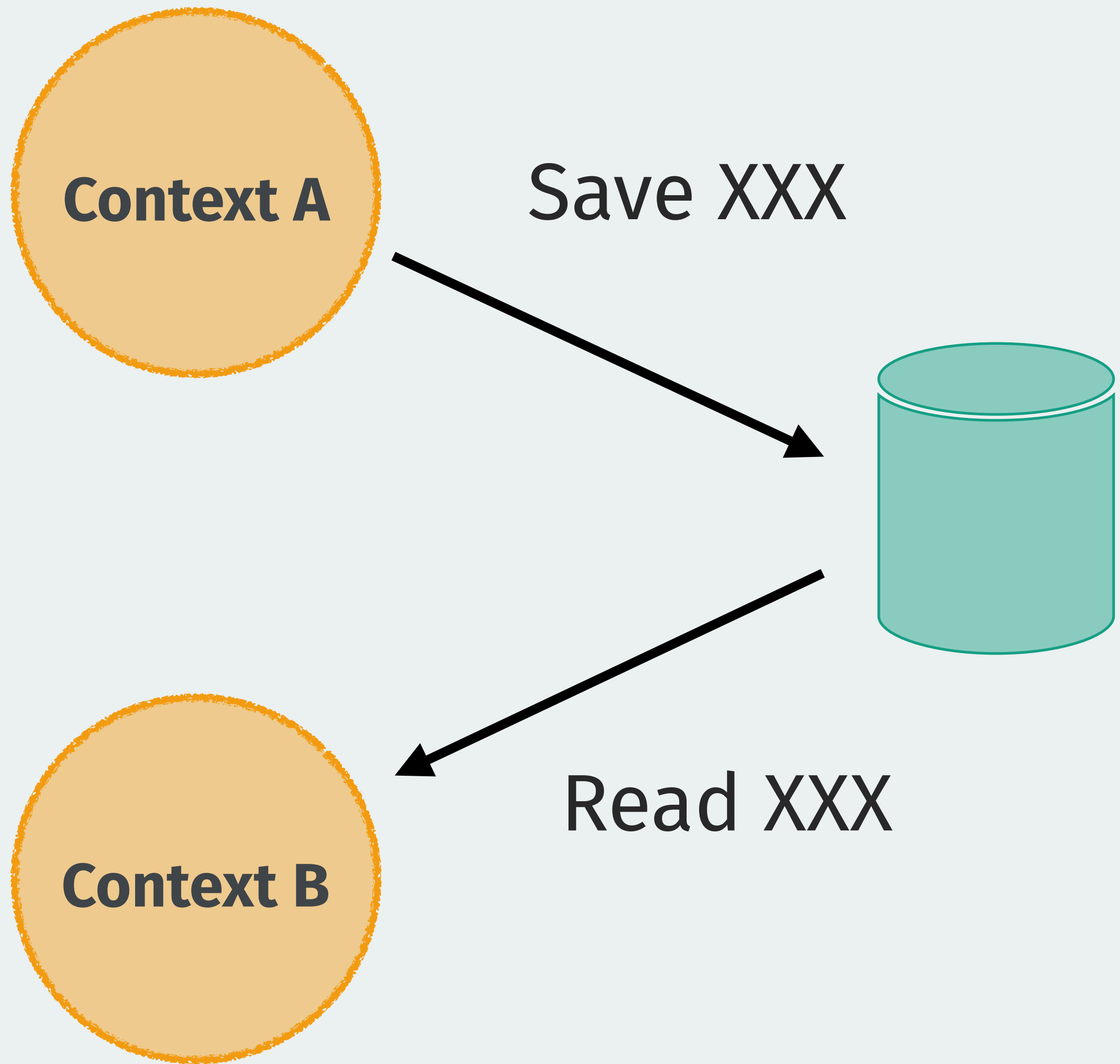
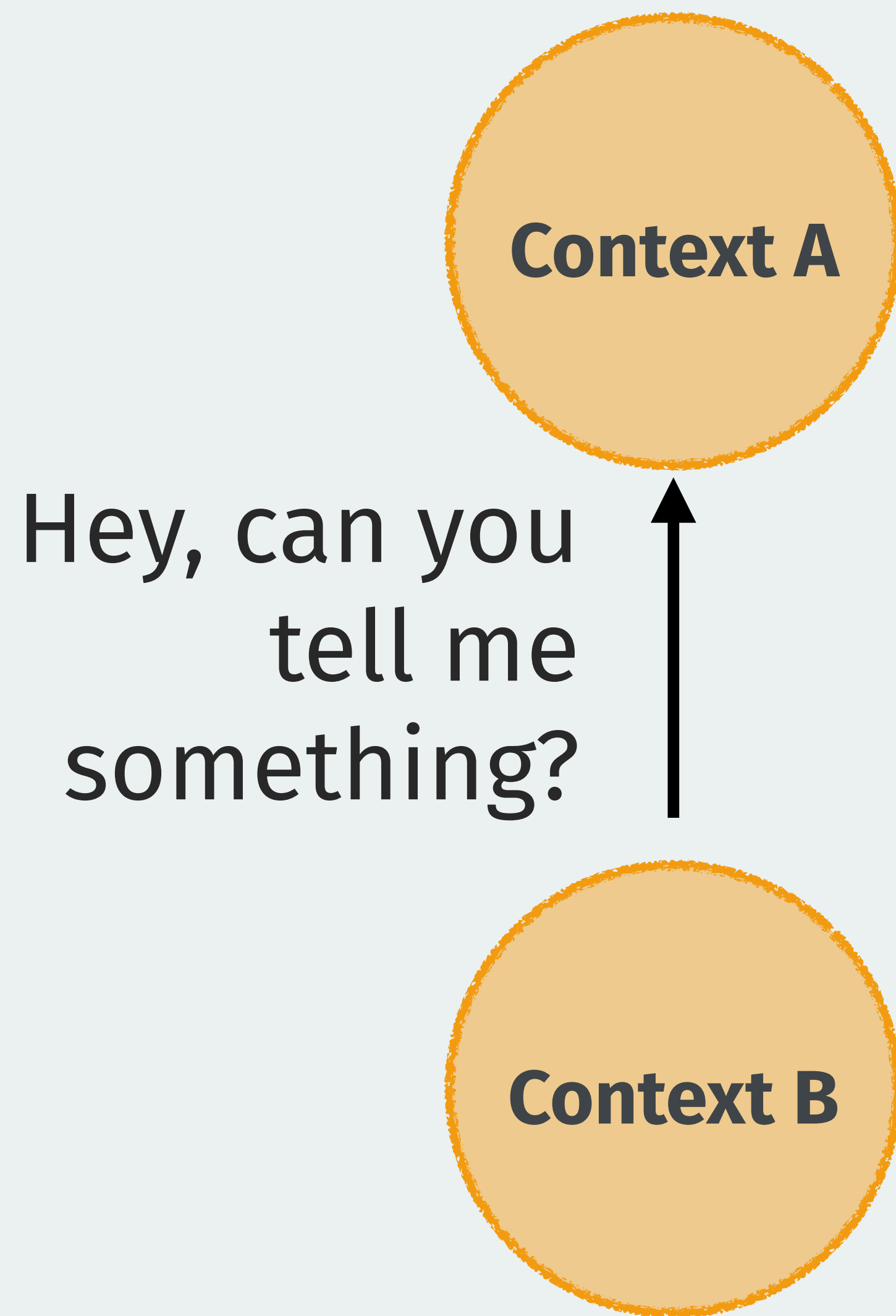
Coupling

Context A

Hey, can you
tell me
something?

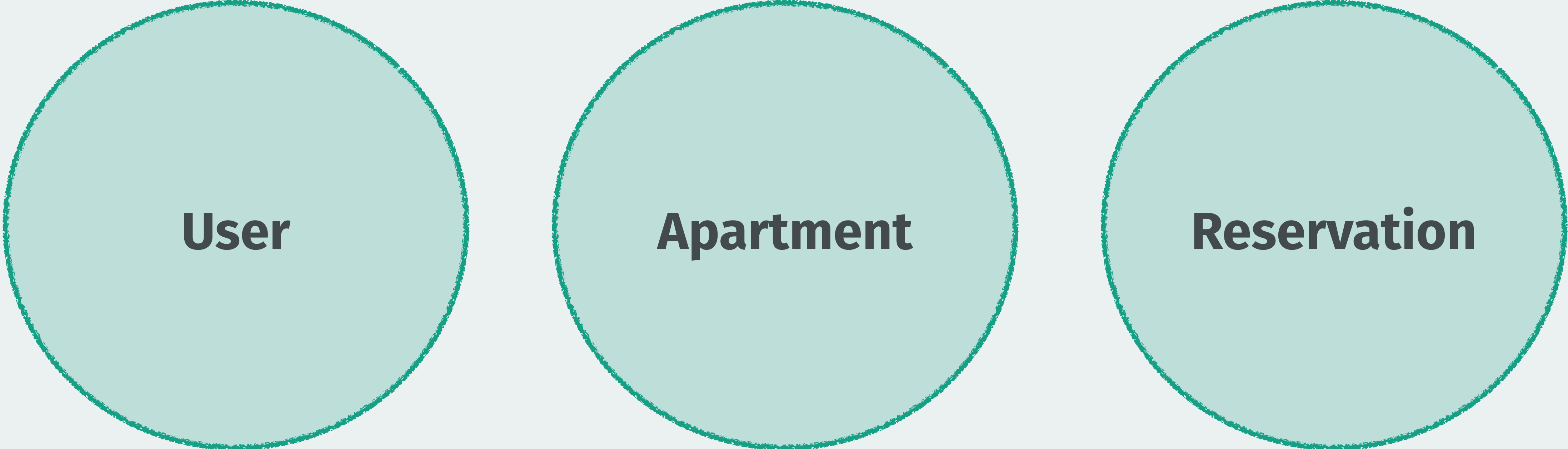
Context B





Few entities,

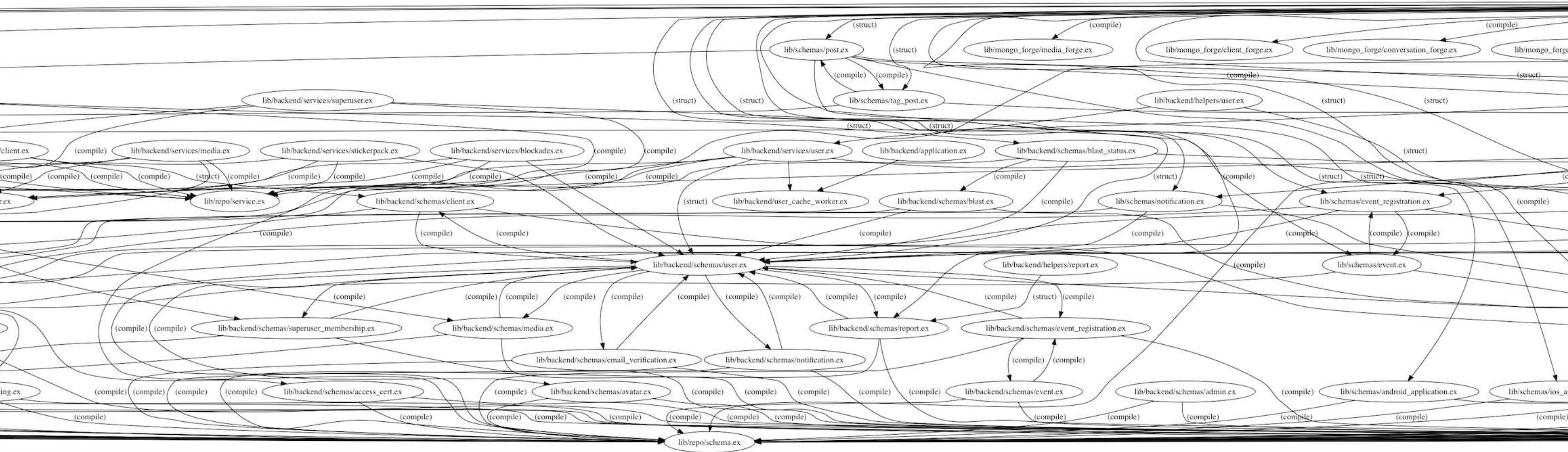
A lot of use cases



User

Apartment

Reservation



Let's take a step back

**What's really
important?**

Start with use cases

**Adding and searching
for apartments**



**Apartment
Management**

AddApartment



AddApartment



**Apartment
Management**

Validate



AddApartment



Validate



Publish



**Apartment
Added**

AddApartment



Validate



Publish



**Apartment
Added**

AddApartment



Validate



Publish



**Apartment
Added**



Subscribe



AddApartment



Validate



Publish



**Apartment
Added**

**Apartment
Search**



Subscribe



AddApartment



Validate



Publish



**Apartment
Added**

**Apartment
Search**



Subscribe

Save



Single responsibility



Validating
apartments



Searching for
apartments

What are events?

Events

- ▶ facts
- ▶ things that happened (in the past)
- ▶ immutable
- ▶ **map closely to business logic**
- ▶ **reduce coupling**

Events

- ▶ facts (things that happened)
- ▶ multiple handlers
- ▶ cannot be rejected
- ▶ past tense

e.g. **ApartmentAdded**

Commands

- ▶ intentions, requests
- ▶ one handler
- ▶ can be rejected
- ▶ present tense

e.g. **AddApartment**

Let's see the code


```
defmodule Airbnb.ApartmentManagement.Logic do

  def add_apartment(%AddApartment{} = command) do
    case validate(command) do
      :ok ->
        event = event_from_command(command)
        [event]

      {:error, changeset} ->
        {:error, changeset}
    end
  end
end

# ...
```

```
defmodule Airbnb.ApartmentManagement do
  @moduledoc "Public interface for managing apartments"

  def add_apartment(%AddApartment{} = command) do
    case ApartmentManagement.Logic.add_apartment(command) do
      {:error, changeset} -> {:error, changeset}

      events when is_list(events) ->
        Enum.each(events, fn event ->
          PubSub.publish("apartments", event)
        end)
    end
  end
end
```

```
defmodule Airbnb.AddingApartments do
  @moduledoc "Public interface for adding apartments"

  def add_apartment(%AddApartment{} = command) do
    case AddingApartments.Logic.add_apartment(command) do
      {:error, changeset} -> {:error, changeset}

      events when is_list(events) ->
        Enum.each(events, fn event ->
          PubSub.publish("apartments", event)
        end)
    end

    :ok
  end
end
```



```
defmodule Airbnb.AddingApartments do
  @moduledoc "Public interface for adding apartments"

  def add_apartment(%AddApartment{} = command) do
    case AddingApartments.Logic.add_apartment(command) do
      {:error, changeset} -> {:error, changeset}

      events when is_list(events) ->
        Enum.each(events, fn event ->
          PubSub.publish("apartments", event)
        end)
    end

    :ok
  end
end
```

```
defmodule Airbnb.ApartmentSearch do
  @behaviour Airbnb.PubSub.Handler

  def handle_event(%ApartmentAdded{} = event) do
    event
    |> Apartment.changeset_from_event()
    |> Airbnb.Repo.insert!(on_conflict: :nothing)

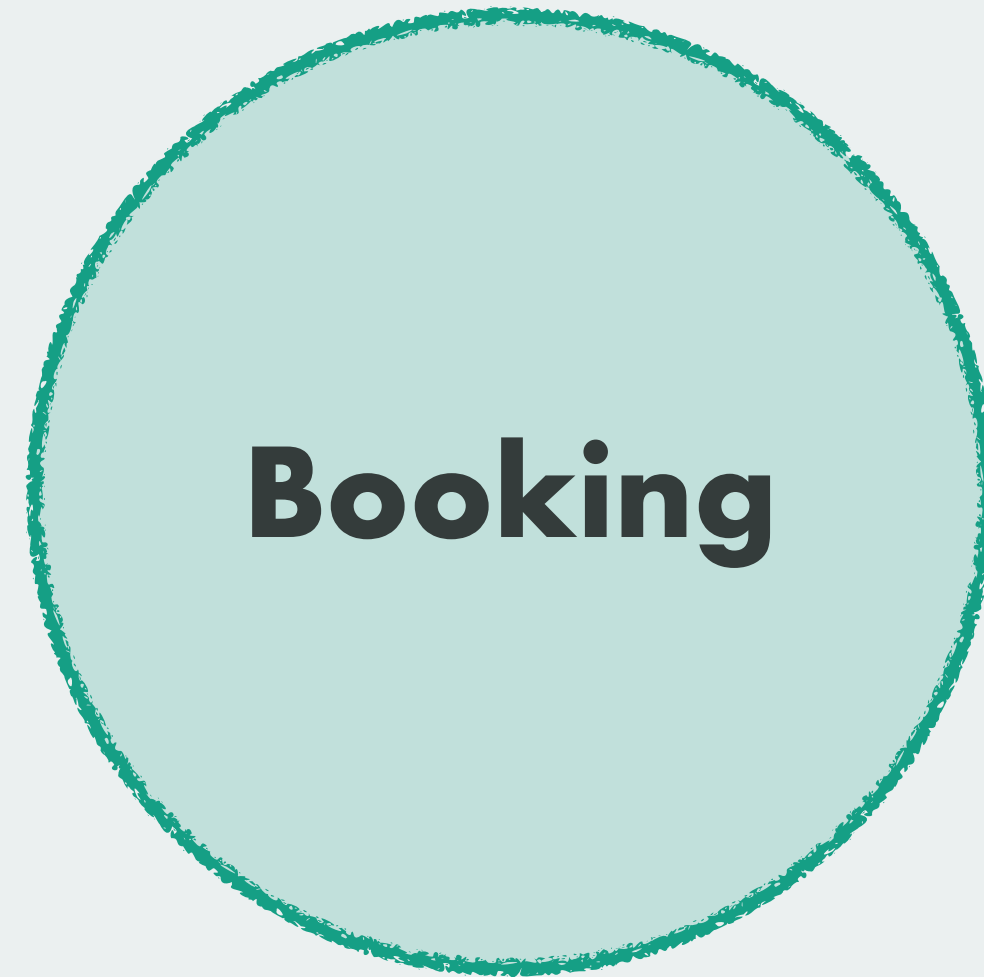
    :ok
  end
end
```

**Pure functions expressing
business rules**

**Side effects only caused
by applying events**



BookApartment



Booking

BookApartment



Booking



BookApartment



Booking

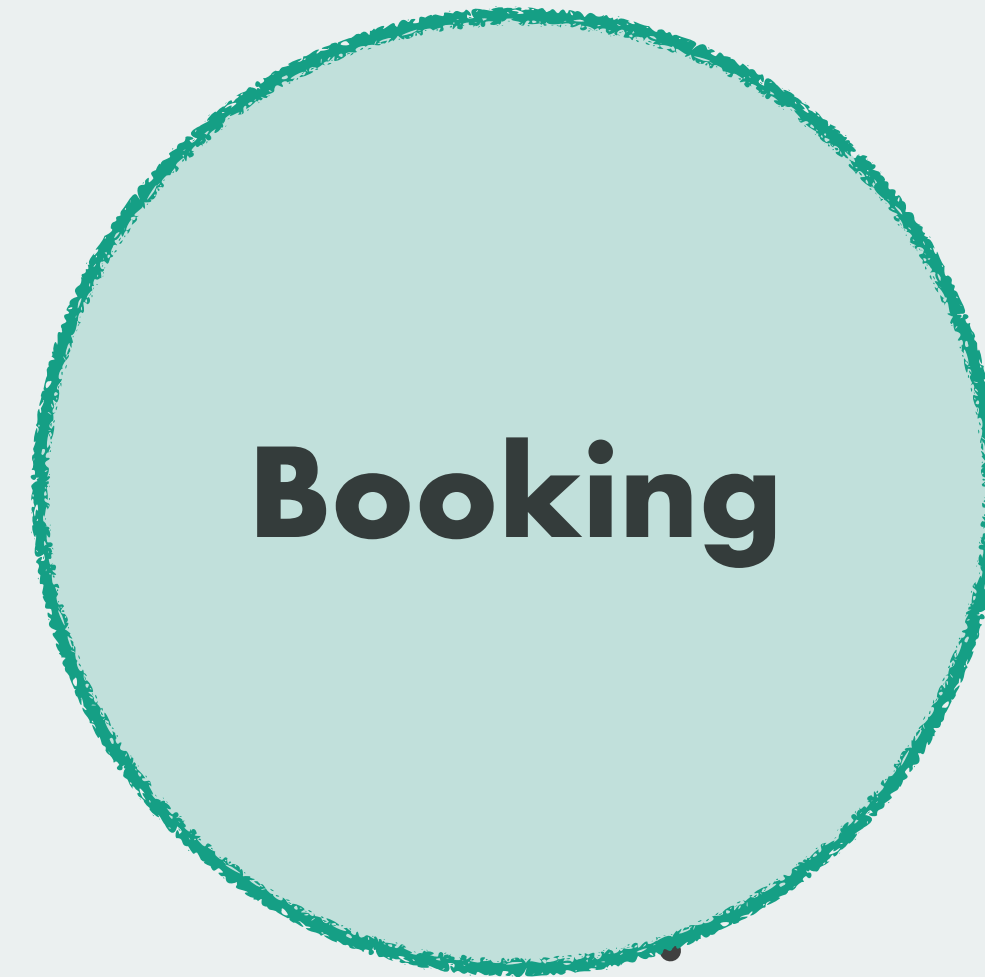


Publish



**Reservation
Requested**

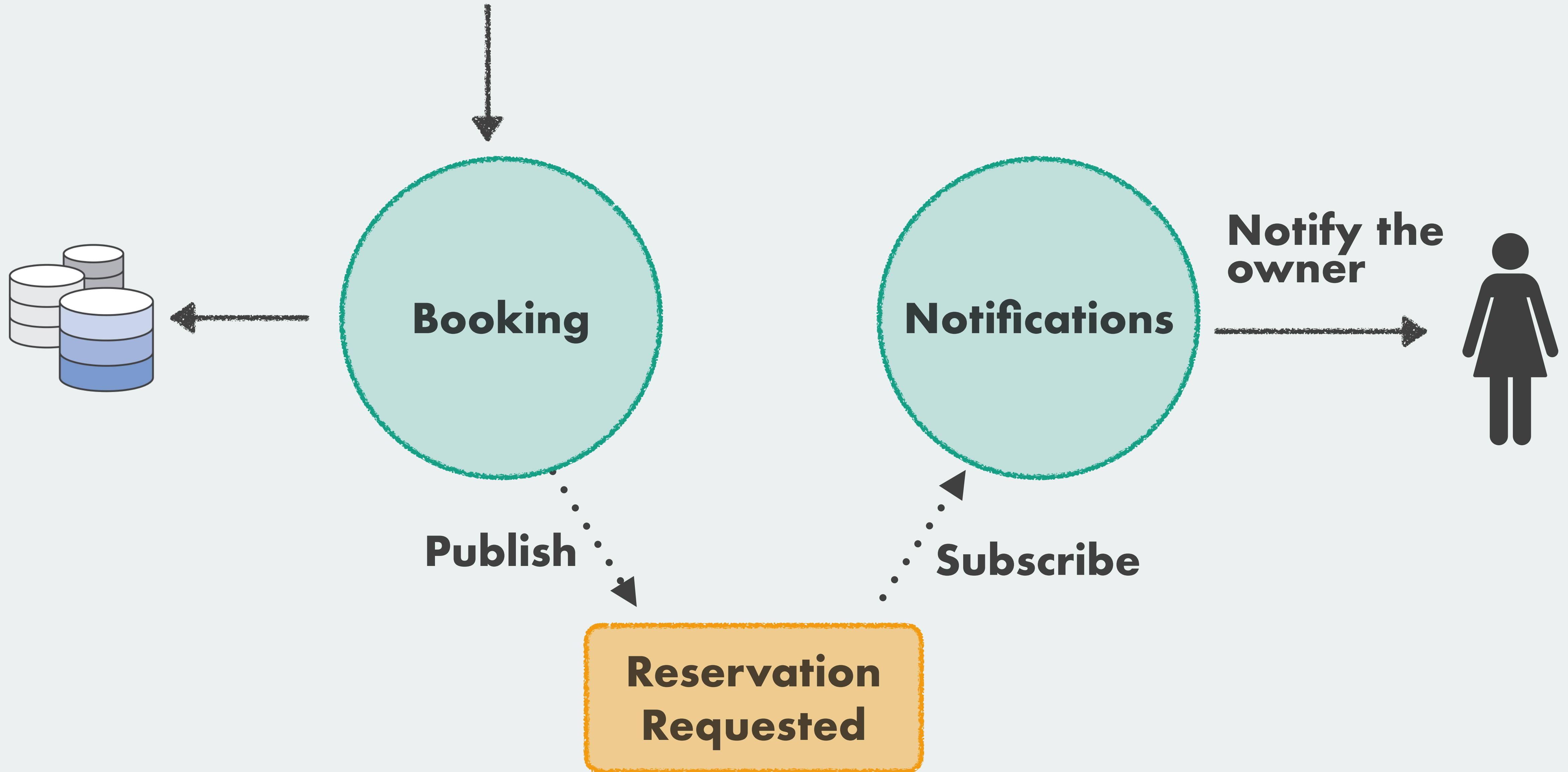
BookApartment



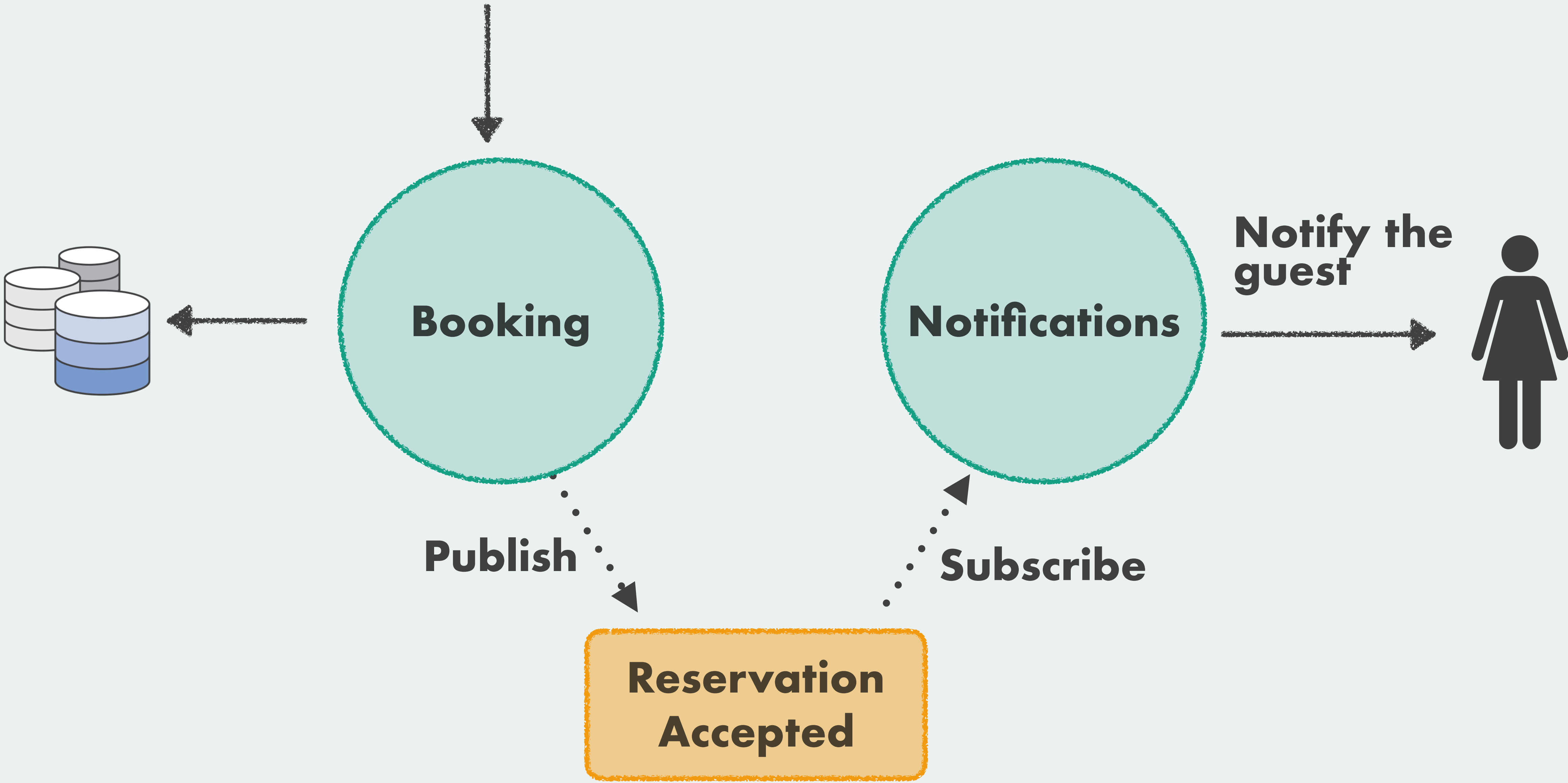
Publish



BookApartment



AcceptReservation



Apartment Management

Responsibility

Validating
commands
(business rules)

Data

Business rules

Data Source

Directly from
the code

Apartment Search

Responsibility

Searching
for
apartments

Data

Available
apartments

Data Source

Apartment-
related events

Payments

Responsibility

Charging the
guests

Data

Billing info

Accepted
requests

Data Source

Data submitted
by the guest

RequestAccepted
event

So, what about PubSub?

```
defmodule Airbnb.PubSub do
  def publish(topic, event) do
    handlers = Application.get_env(:airbnb, :handlers)[topic]

    Enum.each(handlers || [], fn handler ->
      :ok = handler.handle_event(event)
    end)

    :ok
  end
end
```


Using as a dispatcher

`Registry` has a dispatch mechanism that allows developers to implement custom dispatch logic triggered from the caller. For example, let's say we have a duplicate registry started as so:

```
{:ok, _} = Registry.start_link(keys: :duplicate, name: Registry.DispatcherTest)
```

By calling `register/3`, different processes can register under a given key and associate any value under that key. In this case, let's register the current process under the key `"hello"` and attach the `{IO, :inspect}` tuple to it:

```
{:ok, _} = Registry.register(Registry.DispatcherTest, "hello", {IO, :inspect})
```

Now, an entity interested in dispatching events for a given key may call `dispatch/3` passing in the key and a callback. This callback will be invoked with a list of all the values registered under the requested key, alongside the pid of the process that registered each value, in the form of `{pid, value}` tuples. In our example, `value` will be the `{module, function}` tuple in the code above:

```
Registry.dispatch(Registry.DispatcherTest, "hello", fn entries ->
  for {pid, {module, function}} <- entries, do: apply(module, function, [pid])
end)
# Prints #PID<...> where the pid is for the process that called register/3 above
#=> :ok
```

Dispatching happens in the process that calls `dispatch/3` either serially or concurrently in case of multiple partitions (via spawned tasks). The registered processes are not involved in dispatching unless involving

Doesn't really matter

Let's talk about DRY

Don't Repeat Yourself

"Every piece of knowledge must have a single, unambiguous, authoritative representation within a system"

AcceptReservation



Booking



**Apartment
Search**



Publish

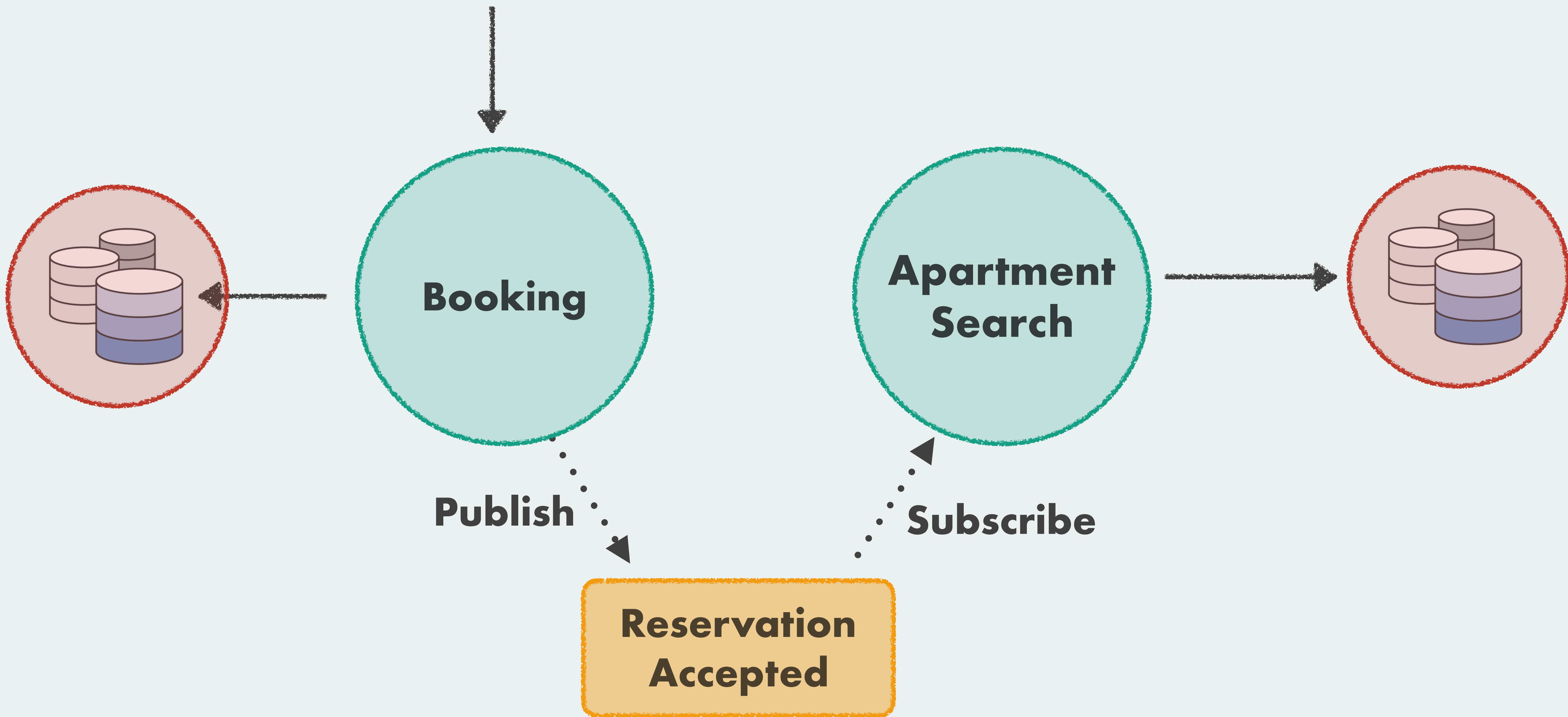


Subscribe



**Reservation
Accepted**

AcceptReservation



Why?

"When the DRY principle is applied successfully, a **modification of any single element of a system does not require a change in other logically unrelated elements.**"

"It's okay to have mechanical, textual duplication (the equivalent of caching values: a repeatable, automatic derivation of one source file from some meta-level description), as long as the authoritative source is well known."

<http://wiki.c2.com/?DontRepeatYourself>

AcceptReservation



Booking



**Apartment
Search**



Publish

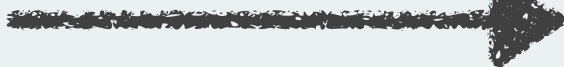


Subscribe



**Reservation
Accepted**

AcceptReservation



PostgreSQL

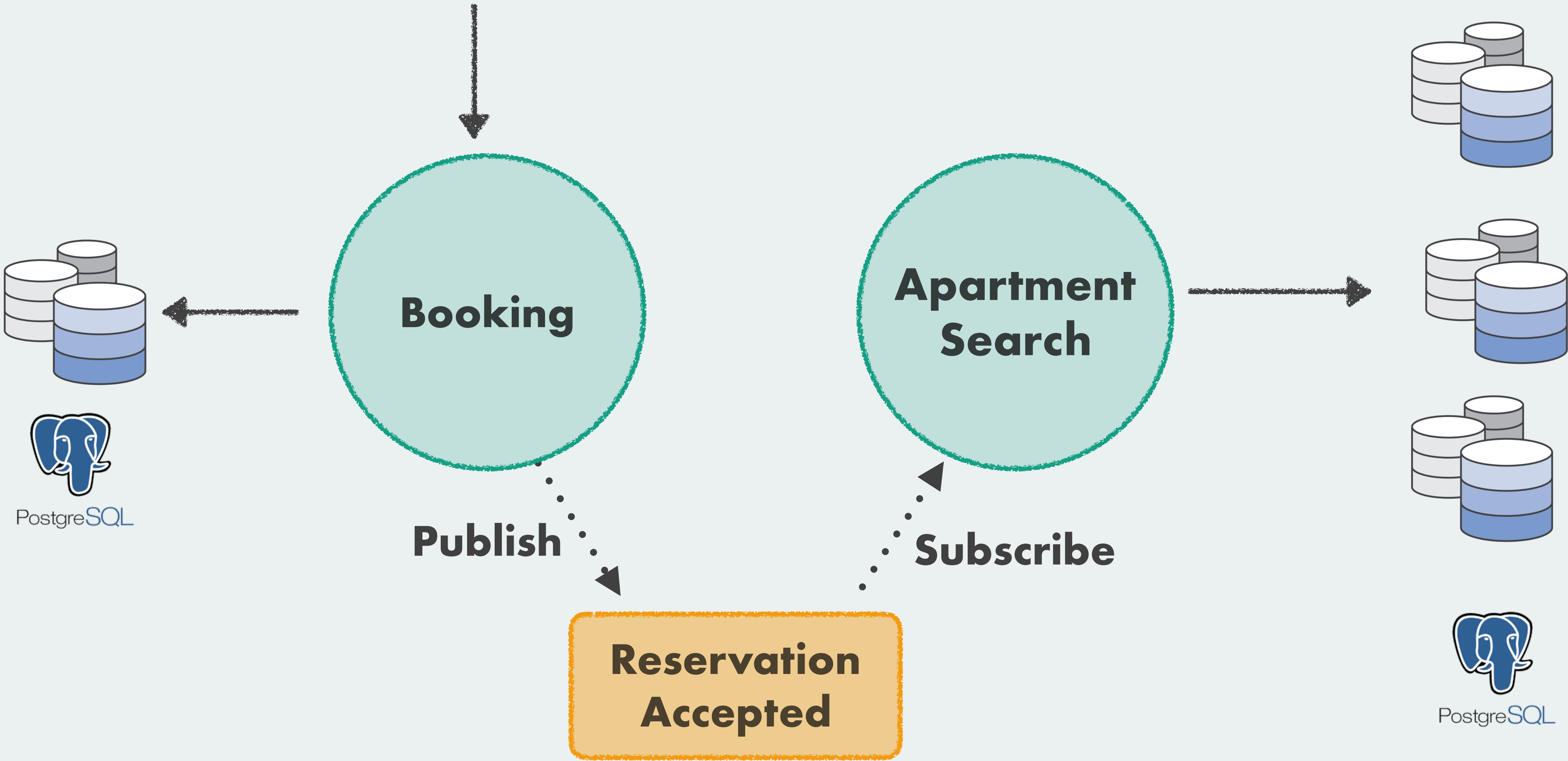
PostgreSQL

Publish

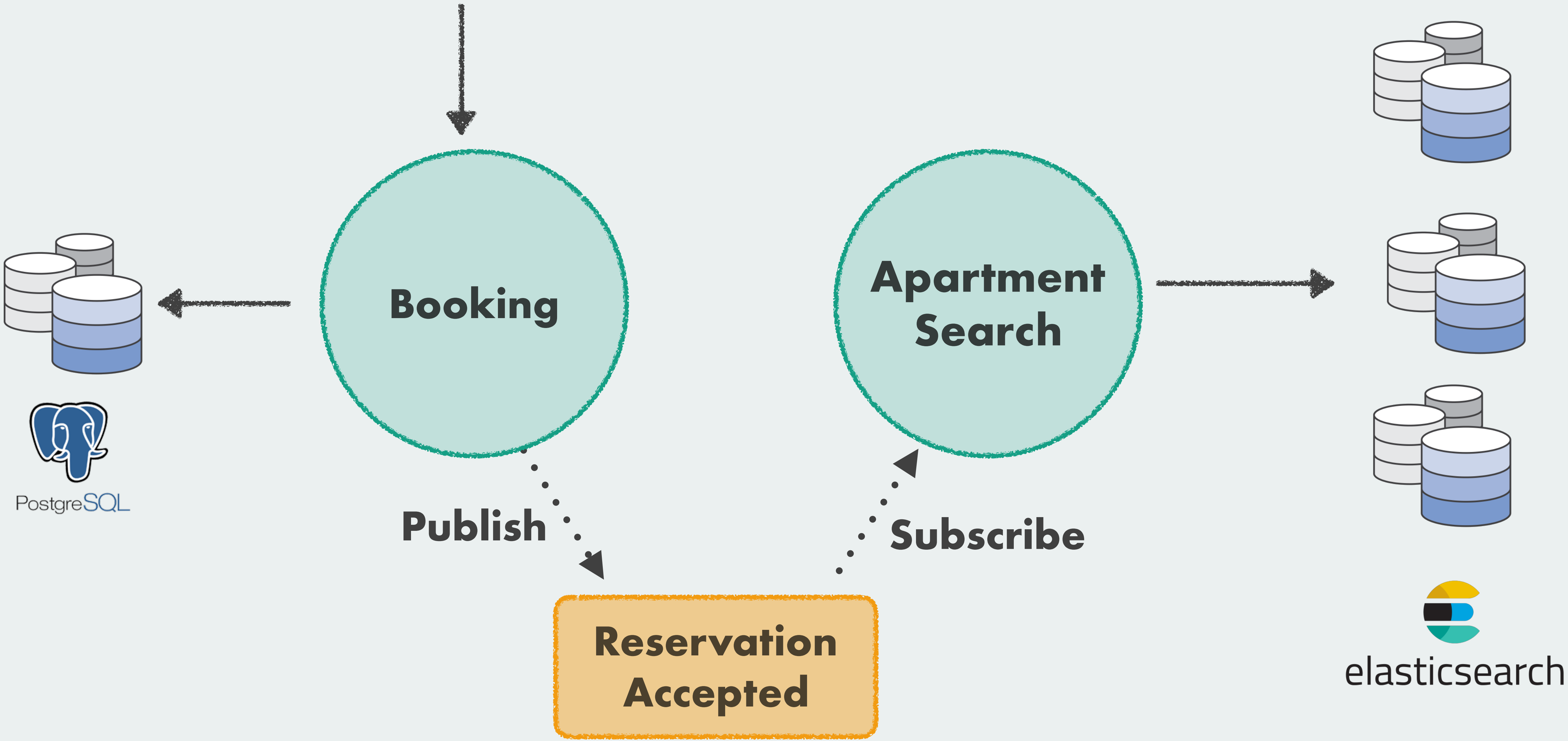
Subscribe



AcceptReservation



AcceptReservation



**Sometimes duplication
allows independent changes**

Summary

Benefits

- ▶ great isolation / loose coupling
- ▶ better domain model
- ▶ better boundaries
- ▶ small components
- ▶ scalable
- ▶ easy to change

Drawbacks

- ▶ more initial work (not always worth it)
- ▶ unfamiliar model
- ▶ (sometimes) eventual consistency /
message loss / ...
- ▶ versioning of events
- ▶ how to discover events?

More cool stuff

- Event Sourcing / CQRS
- Microservices
- Kafka, RabbitMQ, ...
- Storage
- OTP / Message Passing

Think about behaviour

Think about behaviour

- ▶ what does this part do?
- ▶ what does it need to know to do its job?
- ▶ how to provide it with this information?
- ▶ **it's much important than internal data representation**

**Use the tools wisely,
Don't focus on any "rules"**

Thank you!

Maciej Kaszubowski

ElixirConfEU 2018

 **mkaszubowski94**